

Extending memory management tools

I. Motivation

When implementing containers that do not rely on standard allocators it is often necessary to manage memory directly. This paper seeks to fill gaps in the standard library's memory management utilities.

II. Summary

The function template `destroy` calls the destructor for specified elements.

The function template `uninitialized_move` performs move construction of elements over a range of memory, similar to `uninitialized_copy`. `uninitialized_move_n` is also provided.

The function template `uninitialized_value_construct` performs value-construction of objects over a range of memory.

The function template `uninitialized_default_construct` performs default-construction of objects over a range of memory.

III. Discussion

Interface changes proposed in the “range” proposals should be mirrored if both are accepted.

`destroy` first appeared in SGI's Standard Template Library. It is not known by the author why this algorithm was not inherited into the C++ Standard Library in its initial stages. Several responses have preferred that the algorithm be called `destruct`, however, `destroy` maintains convention with SGI and appears to be considered more appropriate use of English.

It is not possible to implement the “no effects” policy for `destroy` so it is specifically excluded from that rule.

The names `uninitialized_value_construct` and `uninitialized_default_construct` explicitly reflect their effects but do not clearly match terminology in other standard library functions. Proposal N3939 has chosen the `_noinit` suffix to denote value vs. default construction. If LEWG prefers this direction then the algorithms could be renamed to `uninitialized_construct` and `uninitialized_construct_noinit`.

Some concern is raised about exception handling with respect to `uninitialized_move`. If a move-constructor throws, moved-from objects may be left in a poorly defined state. Given that algorithm `move` has no special support for this case, it is believed that throwing constructors for this algorithm can be treated similarly. It is believed that the “no effects” wording of this section is sufficient as is.

An additional algorithm, `uninitialized_move_if_noexcept`, could be considered as a resolution to this concern. Given that there is currently no range-based `move_if_noexcept` algorithm, such a solution is not considered here. It is however trivial to implement such an algorithm – simply forwarding to `copy` or `move` as appropriate. The same would hold true for uninitialized algorithms.

IV. Proposed Text

Make the following changes in [specialized.algorithm]:

Modify: In the algorithm `uninitialized_copy` and `uninitialized_move`, the template parameter `InputIterator` is required...

Modify: In the following algorithms other than `destroy`, if an exception is thrown there are no effects.

Add:

```
template<class ForwardIterator>
void destroy(ForwardIterator begin, ForwardIterator end);
```

Effects:

```
typedef typename iterator_traits<ForwardIterator>::value_type __t;
while (begin != end)
{
    begin->~__t();
    ++begin;
}
```

```
template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move(InputIterator first, InputIterator last, ForwardIterator result);
```

Effects:

```
for (; first != last; ++result, ++first)
    ::new (static_cast<void*>(addressof(*result)))
        typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
return result;
```

```
template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move_n(InputIterator first, size_t count, ForwardIterator result);
```

Effects:

```
for (; count>0; ++result, ++first, --count)
    ::new (static_cast<void*>(addressof(*result)))
        typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
return result;
```

```
template<class ForwardIterator>
FwdIt uninitialized_value_construct(ForwardIterator first, ForwardIterator last);
```

Effects:

```
for (; first != last; ++first)
    ::new (static_cast<void*>(addressof(*first)))
        typename iterator_traits<ForwardIterator>::value_type();
return first;
```

```
template<class ForwardIterator>
FwdIt uninitialized_default_construct(ForwardIterator first, ForwardIterator last);
```

Effects:

```
for (; first != last; ++first)
    ::new (static_cast<void*>(addressof(*first)))
        typename iterator_traits<ForwardIterator>::value_type;
return first;
```